

The aim is to design a circuit using the gates already discussed or some other circuits which are in fact derived from the basic gates. As is usual in any engineering design, the number of components used should be minimum to ensure low cost, saving in space, power requirements, etc. There can be two different approaches to the design of combinational circuits. One of these is the traditional method, wherein the given Boolean expression or the truth table is simplified by using standard methods and the simplified expression is realised using the gates. The other method normally does not require any simplification of the logic expression or truth table, instead the complex logic functions available in medium scale integrated circuits (MSI) can be directly used. Computer-aided design (CAD) tools are used for the design using PLDs and FPGAs. Combinational circuit design using the traditional design methods have been discussed below, however, the concepts used in these methods will help in the understanding of design using MSIs, PLDs, and FPGAs, etc.

The following methods can be used to simplify the Boolean functions:

1. Algebraic method,
2. Karnaugh-map technique,
3. Quine–McCluskey method, and
4. Variable entered mapping (VEM) technique.

The algebraic method, the Karnaugh map (K-map) technique, and the Quine-McCluskey method have been discussed here. The K-map is the simplest and most commonly used method. It is a manual method and depends to a great extent upon human intuition. This method can be used conveniently up to six variables beyond which it is very cumbersome.

The Quine–McCluskey method is suitable for computer mechanisation and is seldom used by logic designers manually. The variable entered mapping (VEM) has been discussed in Fletcher^[2] and the interested reader can refer to it.

5.2 STANDARD REPRESENTATIONS FOR LOGIC FUNCTIONS

Logic functions are expressed in terms of logical variables. The values assumed by the logic functions as well as the logic variables are in the binary form. Any arbitrary logic function can be expressed in the following forms:

1. Sum-of-products form (SOP), and
2. Product-of-sums form (POS)

This does not mean that the logic function cannot be written in any other form. It can be written in various forms but the above two forms are conveniently suited in arriving at the standard methods for designing the circuits which will become clear from the following discussions.

Example 5.1

Given the logic equation

$$Y = (A + BC)(B + \overline{C}A) \quad (5.1)$$

- (a) Design a circuit using gates to realise this function.

^[2] Fletcher, W.L., *An Engineering Approach to Digital Design*, Prentice-Hall, Englewood Cliffs, New Jersey, 1980.

- (b) Find out whether it is possible to design the circuit with only one type of gates (NAND or NOR). If yes, design the circuits.
- (c) Find out whether it is possible to simplify this equation. If yes, simplify it.
- (d) Now design the circuit using the simplified expression obtained in part (c).
- (e) Compare the circuits obtained in parts (a), (b), and (d) from the point of view of number of gates, number of inputs for the gates, types of gates, and propagation delay.

Solution

In Eq. (5.1), there are three input logic variables A , B , and C , and Y is the output. The variable C appears as C in one term and as $\bar{C}$ in the other term. A variable in *uncomplemented* or *complemented* form is known as a *literal*.

- (a) A circuit using gates can simply be designed by looking at the expression and finding out the basic gates which can be used to realise the various terms and then connect these gates appropriately. We assume that the signals corresponding to each literal are available, that is the variables are available in uncomplemented as well as the complemented form.
 - (i) The first term (A) has only one literal A and the second term (BC) has two literals B and C . The second term is recognised as an AND operation and can be realised by using a 2-input AND gate. The combination of these two terms is realised by using a 2-input OR gate. The complete realisation of first two terms is shown in Fig. 5.1a.
 - (ii) The third term (B) is again a single literal term and the fourth term ($\bar{C}A$) has two literals. The combination of these two terms is similar to the combination of the first two terms and is realised in a similar way. This realisation is shown in Fig. 5.1b.
 - (iii) Now, the complete realisation is obtained by using a 2-input AND gate with Y_1 and Y_2 as the inputs and the output of this gate will be the required output Y . This realisation is given in Fig. 5.1c. The above design requires three 2-input AND gates and two 2-input OR gates.
- (b) (i) Sum-of-Products Form
Equation (5.1) can be written as

$$\begin{aligned}
 Y &= A(B + \bar{C}A) + (BC)(B + \bar{C}A) && \text{(Theorem 1.9)} \\
 &= AB + A\bar{C}A + BCB + BC\bar{C}A && \text{(Theorem 1.9)} \\
 &= AB + A\bar{C} + BC && \text{(5.3)}
 \end{aligned}$$

Equation (5.3) is obtained from Eq. (5.2) in the following way:

$$\begin{aligned}
 A\bar{C}A &= (A \cdot A) \cdot \bar{C} = A\bar{C} && \text{(Theorem 1.6)} \\
 BCB &= (B \cdot B) \cdot C = BC && \text{(Theorem 1.6)} \\
 BC\bar{C}A &= B \cdot (C \cdot \bar{C}) \cdot A = B \cdot 0 \cdot A && \text{(Theorem 1.8)} \\
 &= 0 && \text{(Theorem 1.4)}
 \end{aligned}$$

This term does not appear in Eq. (5.3), because of Theorem 1.1.

The representation of Eq. (5.3) is known as *sum-of-products* (SOP) form. This can be realised using AND–OR configuration as shown in Fig. 5.2a. This realisation is known as *two-level realisation*. The first level consists of AND gates and the second level consists of the OR gate. By making use of the De-Morgan's theorem (Theorem 1.22) we can write Eq. (5.3) as

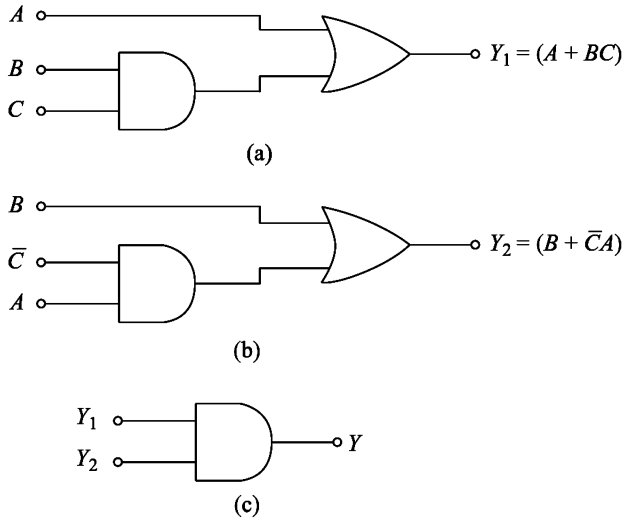


Fig. 5.1 **Logic Circuits for the Realisation of Eq. (5.1)**

$$\begin{aligned}\bar{Y} &= \overline{AB + A\bar{C} + BC} \\ &= \overline{AB} \cdot \overline{A\bar{C}} \cdot \overline{BC}\end{aligned}$$

Or

$$Y = \overline{Y_1 \cdot Y_2 \cdot Y_3} \quad (5.4)$$

Where

$$Y_1 = \overline{AB}$$

$$Y_2 = \overline{A\bar{C}}$$

$$Y_3 = \overline{BC}$$

Equation (5.4) can be realised using NAND gates only. The realisation is given in Fig. 5.2b. This is also a two-level realisation and in this only NAND gates are used. Therefore, if we express the equation in the SOP form we can always design the circuit using only one type of gates (NAND).

(ii) Product-of-Sums Form

It is possible to represent Eq. (5.1) in another form.

$$\begin{aligned}Y &= (A + B)(A + C)(B + \bar{C})(B + A) && \text{(Theorem 1.10)} \\ &= (A + B)(A + C)(B + \bar{C}) && \text{(Theorem 1.6)}\end{aligned} \quad (5.5)$$

The representation of Eq. (5.5) is known as *Product-of-sums* (POS) form. This can be realised using OR-AND gates as shown in Fig. 5.3a. This is also a two-level realisation

Using De-Morgan's theorem (Theorem 1.21), we can write Eq. (5.5) as

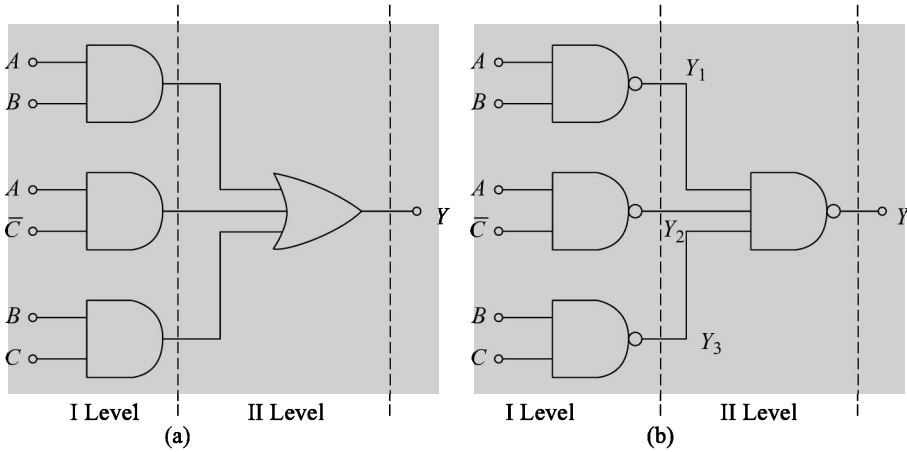


Fig. 5.2 Realisation of Eq. (5.3) Using (a) AND-OR (b) NAND-NAND

$$\begin{aligned}\bar{Y} &= \overline{(A + B)(A + C)(B + \bar{C})} \\ &= \overline{(A + B)} + \overline{(A + C)} + \overline{(B + \bar{C})}\end{aligned}$$

Or

$$Y = \bar{Y}_A + \bar{Y}_B + \bar{Y}_C \quad (5.6)$$

Where

$$Y_A = \overline{A + B}$$

$$Y_B = \overline{A + C}$$

$$Y_C = \overline{B + \bar{C}}$$

Equation (5.6) can be realised using NOR gates only. This realisation is given in Fig. 5.3b which is a two-level realisation. Hence, if we express the equation in POS form we can always design the circuit using only one type of gates (NOR).

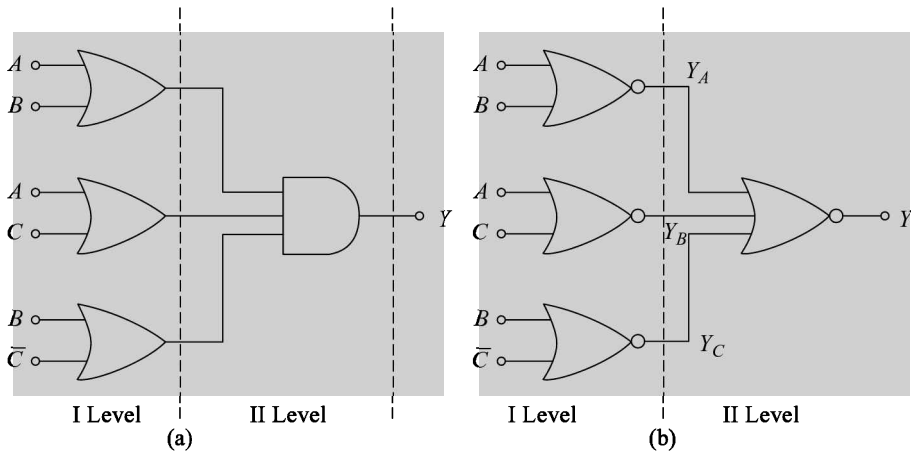


Fig. 5.3 Realisation of Eq. (5.5) Using (a) OR-AND (b) NOR-NOR

(c) (i) Simplification of Eq. (5.3),

$$\begin{aligned} Y &= AB + A\bar{C} + BC \\ &= BC + A\bar{C} \quad (\text{Theorem 1.19}) \end{aligned} \quad (5.7)$$

(ii) Simplification of Eq. (5.5),

$$\begin{aligned} Y &= (A + B)(A + C)(B + \bar{C}) \\ &= (A + C)(B + \bar{C}) \quad (\text{Theorem 1.20}) \end{aligned} \quad (5.8)$$

(d) Realisation of Eqs (5.7) and (5.8) are given in Figs 5.4a and 5.4b, respectively.

(e) The gate requirements corresponding to parts (a), (b), and (d) are given in Table 5.1. The realisation of part (a) needs maximum number of gates. Also, it is a three-level realisation which increases the propagation delay time which is same as decrease in speed of operation. Realisations corresponding to parts (b) and (d) are very useful since only one type of gates (NAND/NOR) are required which is very convenient to use when we use ICs because a number of similar gates are available in the same package. Realisation corresponding to part (d) requires minimum number of gates. Therefore, the simplification of logic expressions is very useful

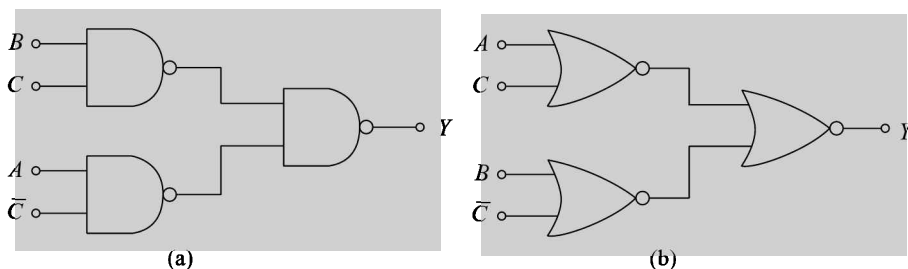


Fig. 5.4 Realisation of (a) Equation (5.7) (b) Equation (5.8)

Table 5.1 Gate Requirements

Part (a)	Part (b)		Part (d)
	AND-OR/NAND-NAND	OR-AND/NOR-NOR	
3, 2-input AND	3, 2-input AND	3, 2-input OR	3, 2-input NAND or 3, 2-input NOR
2, 2-input OR	1, 3-input OR or 3, 2-input NAND 1, 3-input NAND	1, 3-input AND or 3, 2-input NOR 1, 3-input NOR	

We notice that in the SOP form of Eq. (5.3) and the POS form of Eq. (5.5), all the individual terms do not involve all the three literals. If each term in SOP and POS forms contains all the literals then these are known as *canonical* SOP and POS, respectively. Each individual term in canonical SOP form is called as *minterm* and in *canonical* POS form as *maxterm*. The usefulness of the minterm and maxterm representations will become clear from the discussion which follows. SOP form can be converted to canonical SOP by ANDing the terms in the expression with terms formed by ORing the variable and its complement which are not present in that term. For example for a three-variable expression with variables A , B , and C , if there is a term A , where B and C variables are missing, then we form two terms $(B + \bar{B})$ and $(C + \bar{C})$ and AND them with A . Therefore, we get $A \cdot (B + \bar{B}) \cdot (C + \bar{C}) = ABC + \bar{A}BC + A\bar{B}C + \bar{A}\bar{B}C$.

Example 5.2

Convert Eq. (5.3) into canonical SOP form.

Solution

$$\begin{aligned} Y &= AB(C + \bar{C}) + A\bar{C}(B + \bar{B}) + BC(A + \bar{A}) \\ &= ABC + \bar{A}BC + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} + ABC + \bar{A}BC \\ &= ABC + \bar{A}BC + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} \end{aligned} \quad \begin{array}{l} \text{(Theorem 1.5)} \\ (5.9) \end{array}$$

Similarly, POS form can be converted to canonical POS by ORing the terms in the expression with terms formed by ANDing the variable and its complement which are not present in that term. For example for a three-variable expression with variables A , B , and C , if there is a term A where B and C variables are missing, then we form two terms $B \times \bar{B}$ and $C \times \bar{C}$ and OR A with these terms. Therefore, we get

$$\begin{aligned} A + B\bar{B} + C\bar{C} &= (A + B\bar{B} + C)(A + B\bar{B} + \bar{C}) \quad \text{(Theorem 1.10)} \\ &= (A + B + C)(A + \bar{B} + C)(A + B + \bar{C})(A + \bar{B} + \bar{C}) \end{aligned} \quad (5.10)$$

Example 5.3

Convert Eq. (5.5) into canonical POS form.

Solution

$$\begin{aligned} Y &= (A + B + C\bar{C})(A + B\bar{B} + C)(A\bar{A} + B + \bar{C}) \\ &= (A + B + C)(A + B + \bar{C})(A + B + C)(A + \bar{B} + C) \\ &\quad (A + B + \bar{C})(\bar{A} + B + \bar{C}) \quad \text{(Theorem 1.10)} \\ &= (A + B + C) \cdot (A + B + \bar{C}) \cdot (A + \bar{B} + C) \cdot (\bar{A} + B + \bar{C}) \quad \text{(Theorem 1.6)} \end{aligned} \quad (5.11)$$